Addressing mode = Manner in which Operand is given in instruction.

Addressing modes are listed below :
1.Immidiate Mode (Data/addr. is given immediately in instruction)
2.Register Addressing Mode (Data is stored at Register)
3. Direct Addressing Mode (Addr. is given in instruction)
4. Indirect Addressing Mode (Data is present at mem. loc. pointed by reg. pair)
5. Implied Addressing Mode (opcode itself contains the operand)

8085 Addressing Modes & Interrupts

Now let us discuss the addressing modes in 8085 Microprocessor.

Addressing Modes in 8085

These are the instructions used to transfer the data from one register to another register, from the memory to the register, and from the register to the memory without any alteration in the content. Addressing modes in 8085 is classified into 5 groups –

Immediate addressing mode

In this mode, the 8/16-bit data is specified in the instruction itself as one of its operand. **For example:** MVI K, 20F: means 20F is copied into register K.

Register addressing mode

In this mode, the data is copied from one register to another. **For example:** MOV K, B: means data in register B is copied to register K.

Direct addressing mode

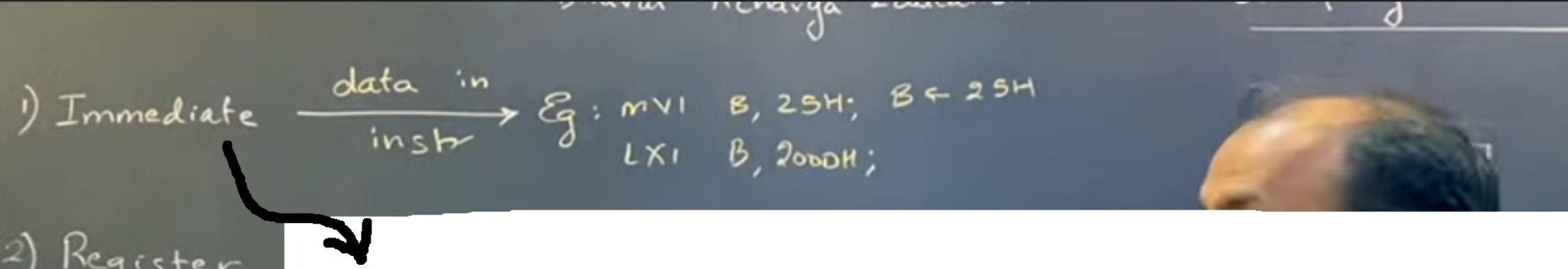
In this mode, the data is directly copied from the given address to the register. **For example:** LDB 5000K: means the data at address 5000K is copied to register B.

Indirect addressing mode

In this mode, the data is transferred from one register to another by using the address pointed by the register. **For example:** MOV K, B: means data is transferred from the memory address pointed by the register to the register K.

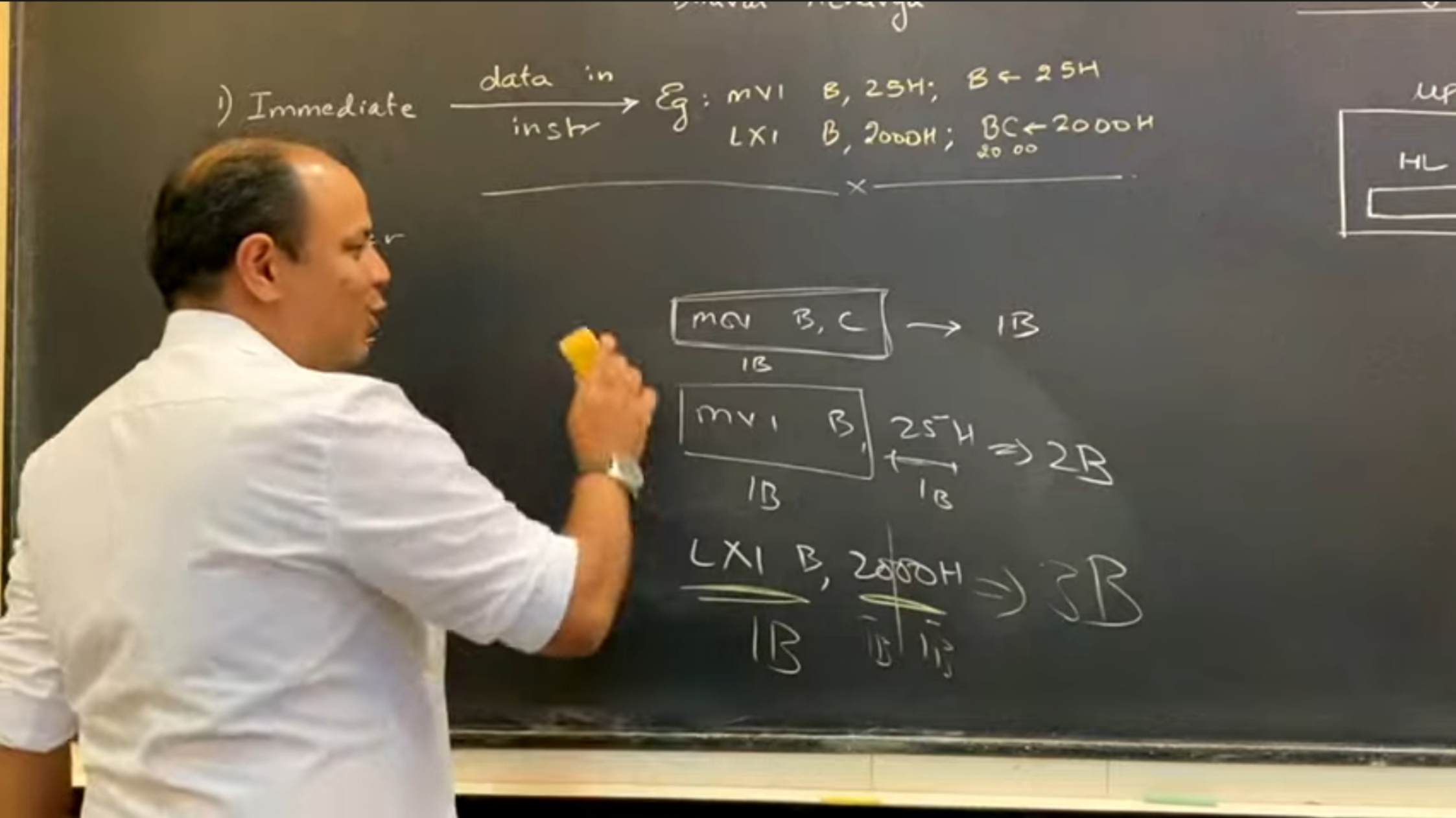
Implied addressing mode

This mode doesnt require any operand; the data is specified by the opcode itself. **For example:** CMP.



Data is **directly** given **in instruction** and loaded into the register.

if **X** is there, it **implies Register Pair** instead of a single Register.
so, LXI B,2000H will store 20H in B and 00H in C.


1) Immediate
data in
inst
Eg: MVI B, 25H; B ← 25H
LXI B, 2000H; BC ← 2000H
20 00
HL
MOV B, C
1B
→ 1B
MVI B, 25H
1B
1B
⇒ 2B
LXI B, 2000H
1B
1B 1B
⇒ 3B

1. Zero-Operand Instructions (Implicit)

Some opcodes are "self-contained." The CPU already knows what to act on because it's implied by the command.

- **Example:** `CMA` (Complement Accumulator).
- **Structure:** `[ Opcode ]`
- **Total Size:** 8 bits.

2. 8-Bit Operand Instructions (Immediate Data)

Here, the opcode says "Load a number," and the very next byte in memory is that specific number.

- **Example:** `MVI A, 05H` (Move Immediate the value 5 into Register A).
- **Structure:** `[ Opcode ] [ 05H ]`
- **Total Size:** 16 bits (2 bytes).

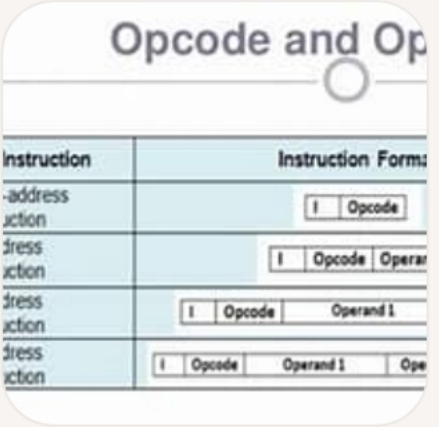
3. 16-Bit Operand Instructions (Addresses)

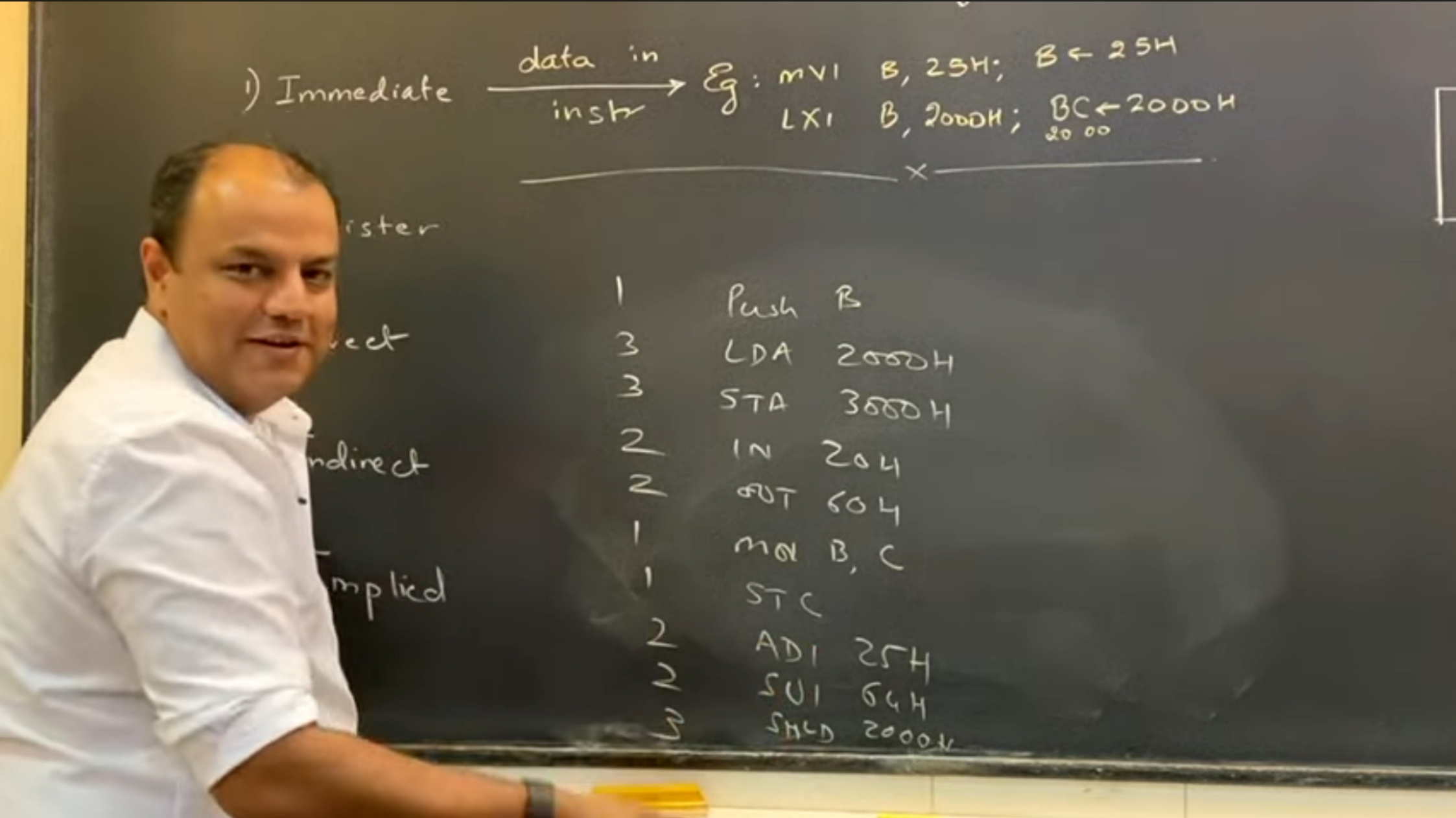
When you need to reference a specific location in memory, 8 bits isn't enough (it only covers 256 spots). You need 16 bits to point to a wider range of memory.

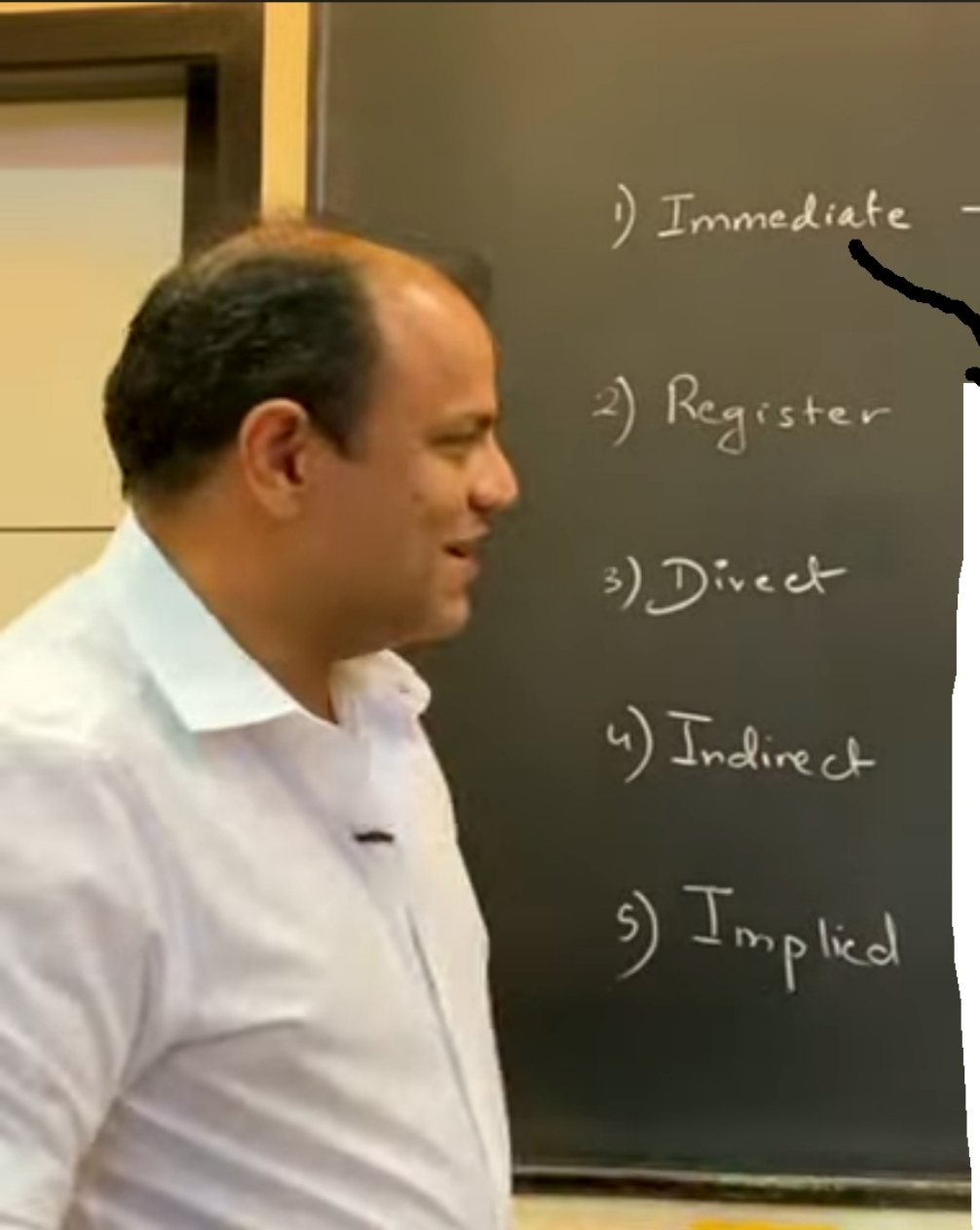
- **Example:** `LDA 2050H` (Load Accumulator with the contents of memory address 2050).
- **Structure:** `[ Opcode ] [ 50H ] [ 20H ]` *(Note: Many CPUs use "Little Endian" format, storing the low byte first).*
- **Total Size:** 24 bits (3 bytes).

operation, opcode and operand in 8085

In the context of the 8085 microprocessor, **operation** refers to the specific task the processor is to perform, such as addition, subtraction, or memory access. **Opcode** is the binary code that represents the operation to be performed, and **operand** is the data or memory location that the operation will affect or be affected by. These components work together to create instructions that the processor can execute to perform tasks as programmed. bimstudies.com +5




1) Immediate
data in
inst
Eg: MVI B, 25H; B ← 25H
LXI B, 2000H; BC ← 2000H
20 00
ister
ect
ndirect
mplied
1 Push B
3 LDA 2000H
3 STA 3000H
2 IN 20H
2 OUT 60H
1 MOV B, C
1 STC
2 ADI 25H
2 SUI 64H
3 LHLD 2000H

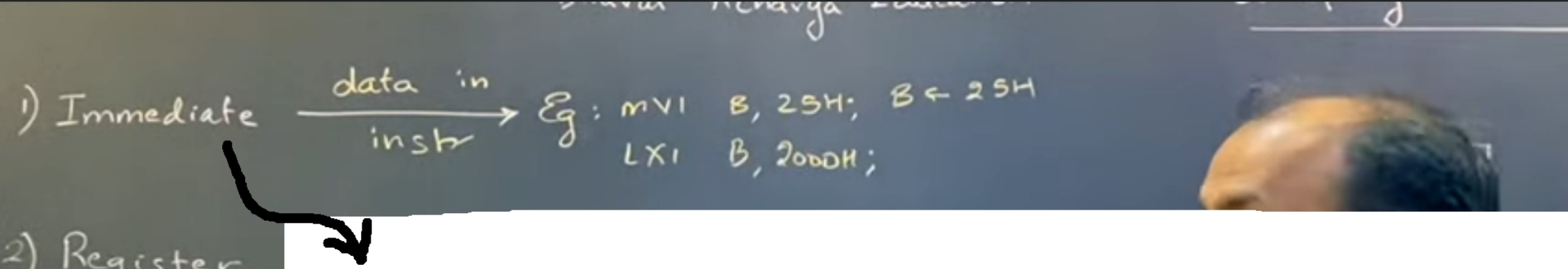


Advantage :

Execution is Fast as number is already given in instruction.

Disadvantage :

Instruction will always be of 2 or 3 bytes as we are giving number along with the instruction.



Data is **directly** given **in instruction** and loaded into the register.

if **X** is there, it **implies Register Pair** instead of a single Register.
so, LXI B,2000H will store 20H in B and 00H in C.

Bharat Achaya Education

Concept of M - memory Pointer

1) Immediate — data in instr → Eg: MVI B, 25H; B ← 25H
LXI B, 2000H; BC ← 2000H (20 00)

2) Register — data in reg → Eg: MOV B, C; B ← C
INR B; B ← B+1
INX B; BC ← BC+1

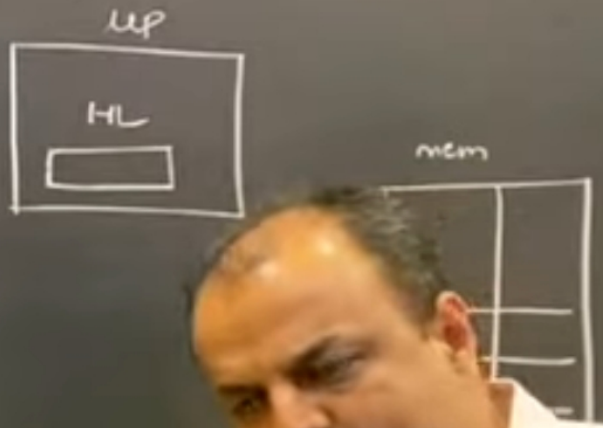
3) Direct

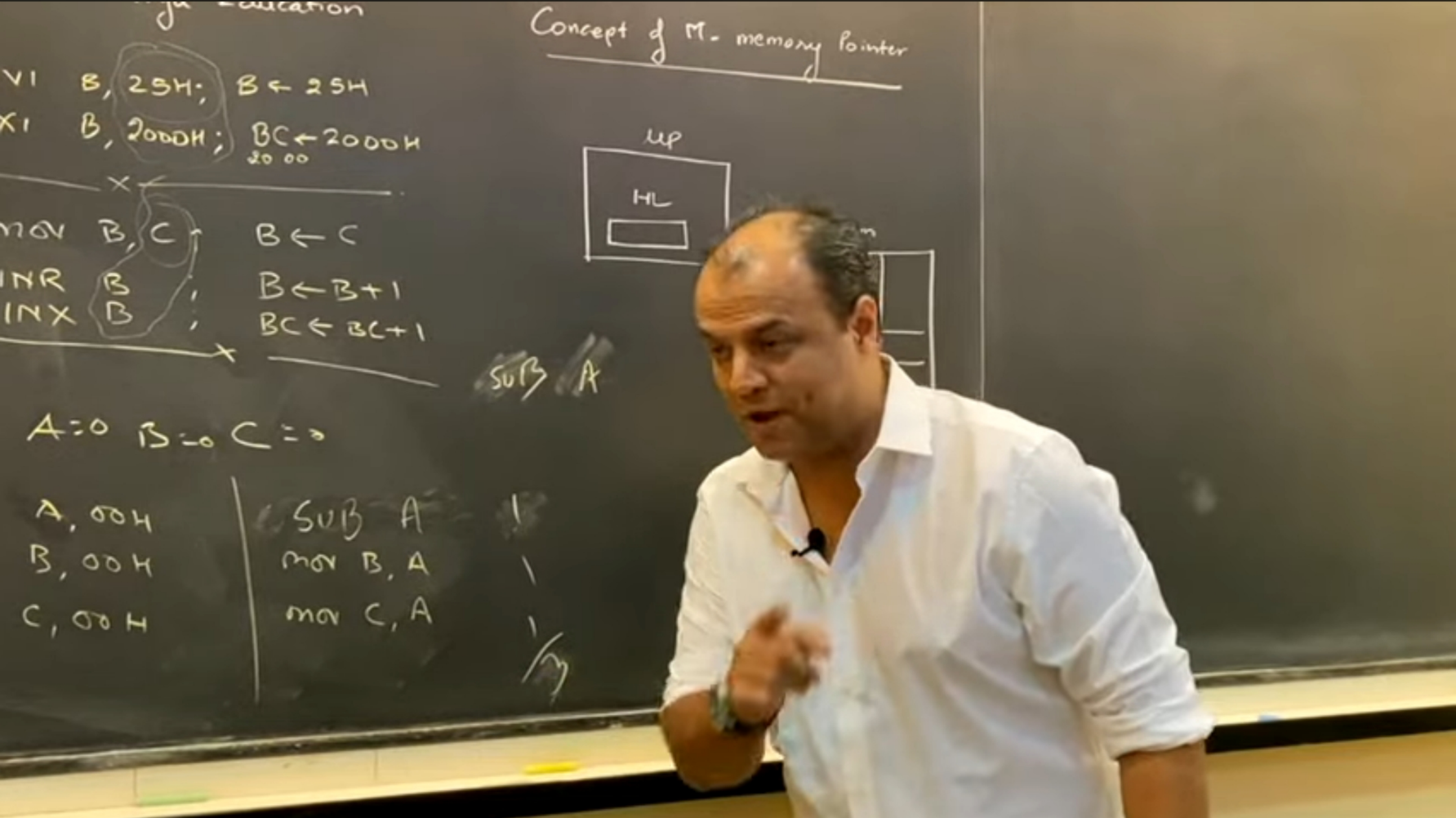
4) Indirect

5) Implied

A:0 B:0 C:0

2 MVI A, 00H	MVI A, 00H 2
2 MVI B, 00H	MOV B, A 1
2 MVI C, 00H	MOV C, A 1
6	4




Concept of M- memory Pointer
VI B, 25H; B ← 25H
XI B, 2000H; BC ← 2000H
20 00
MOV B, C B ← C
INR B B ← B+1
INX B BC ← BC+1
SUB A
A=0 B=0 C=0
A, 00H
B, 00H
C, 00H
SUB A
mov B, A
mov C, A
UP
HL

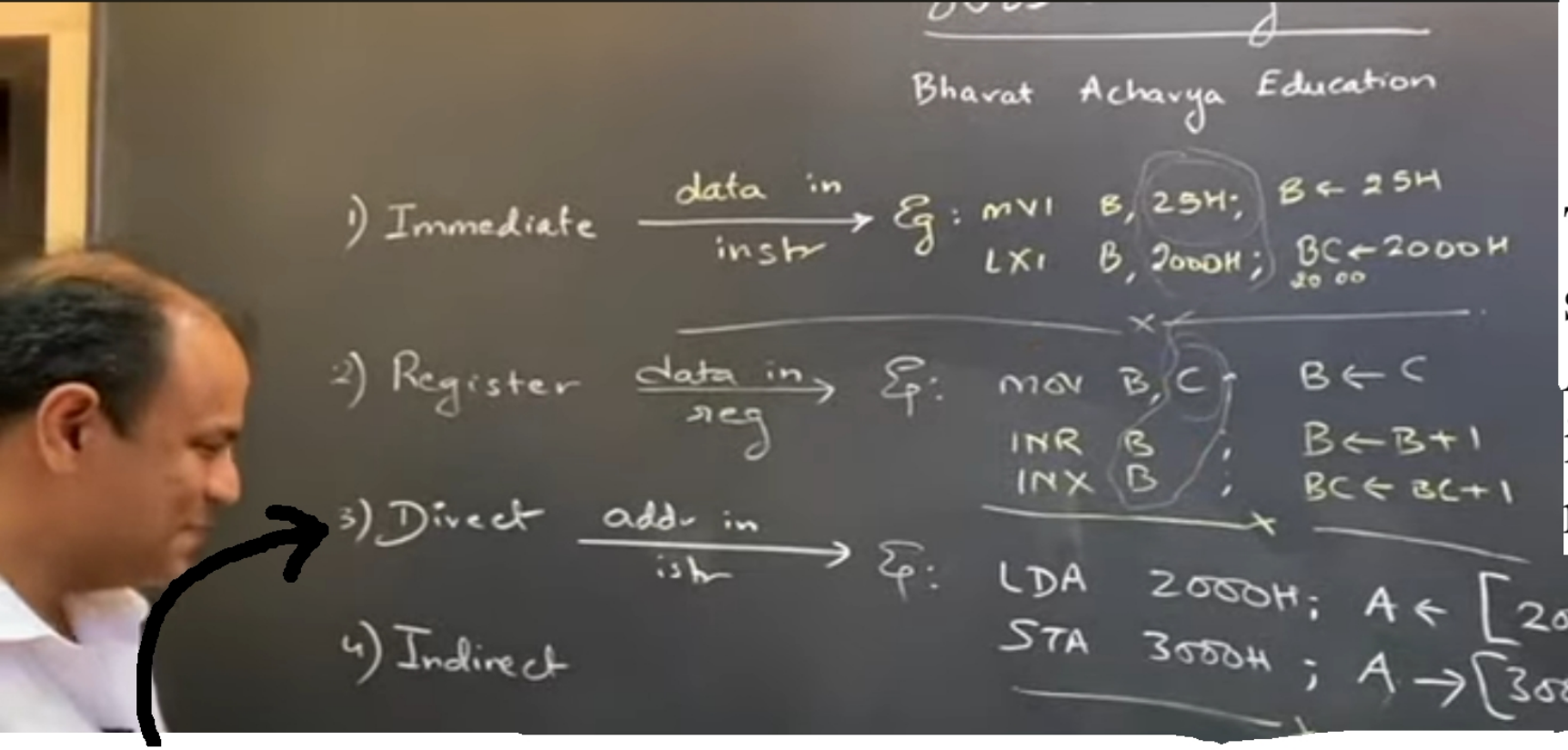
1) Immediate → data in instr → Eg: MVI B, 25H; B ← 25H
LXI B, 2000H; BC ← 2000H (20 00)

2) Register → data in reg → Eg: MOV B, C ; B ← C
INR B ; B ← B+1
INX B ; BC ← BC+1

3) Direct → addr in instr → Eg: LDA 2000H; A ←
STA 3000H ; A

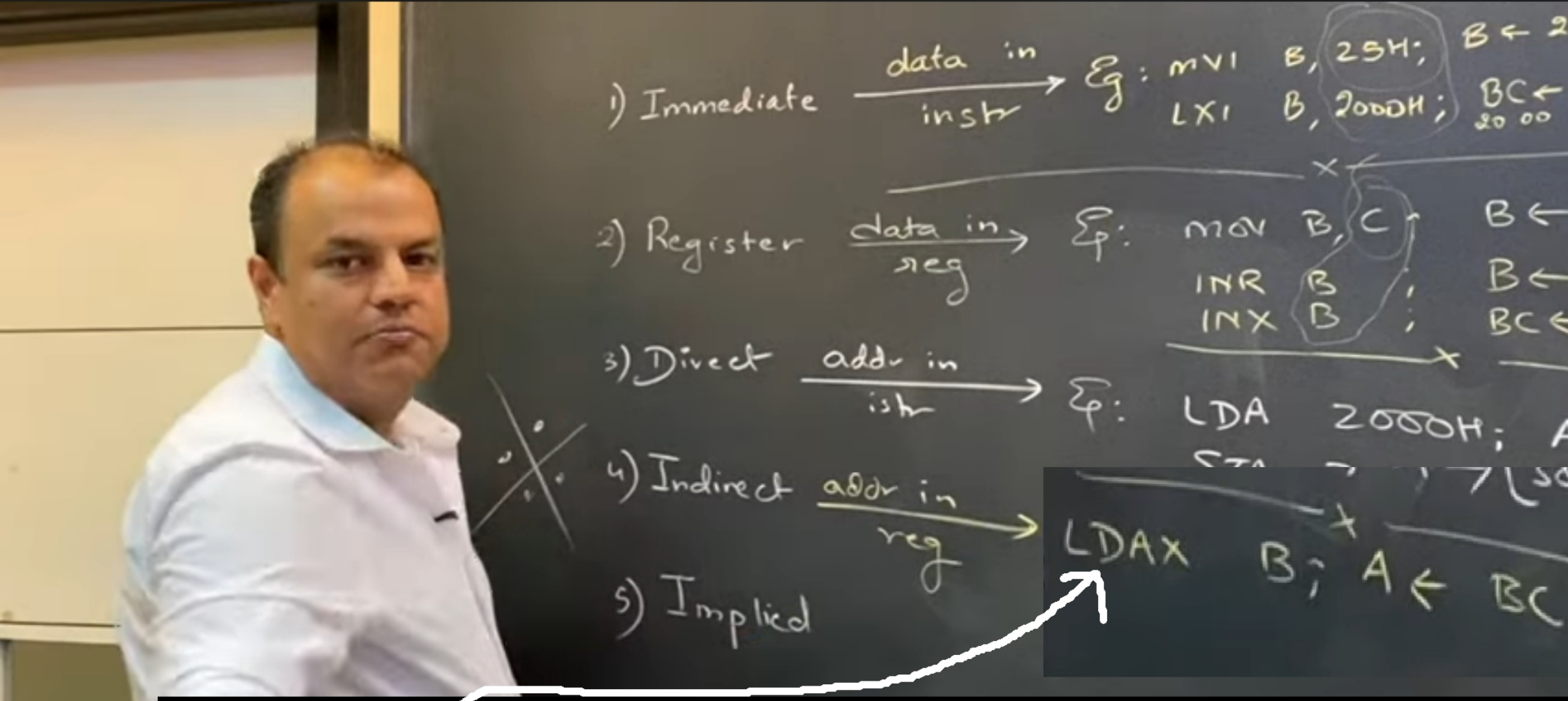
4) Indirect

5) Implied

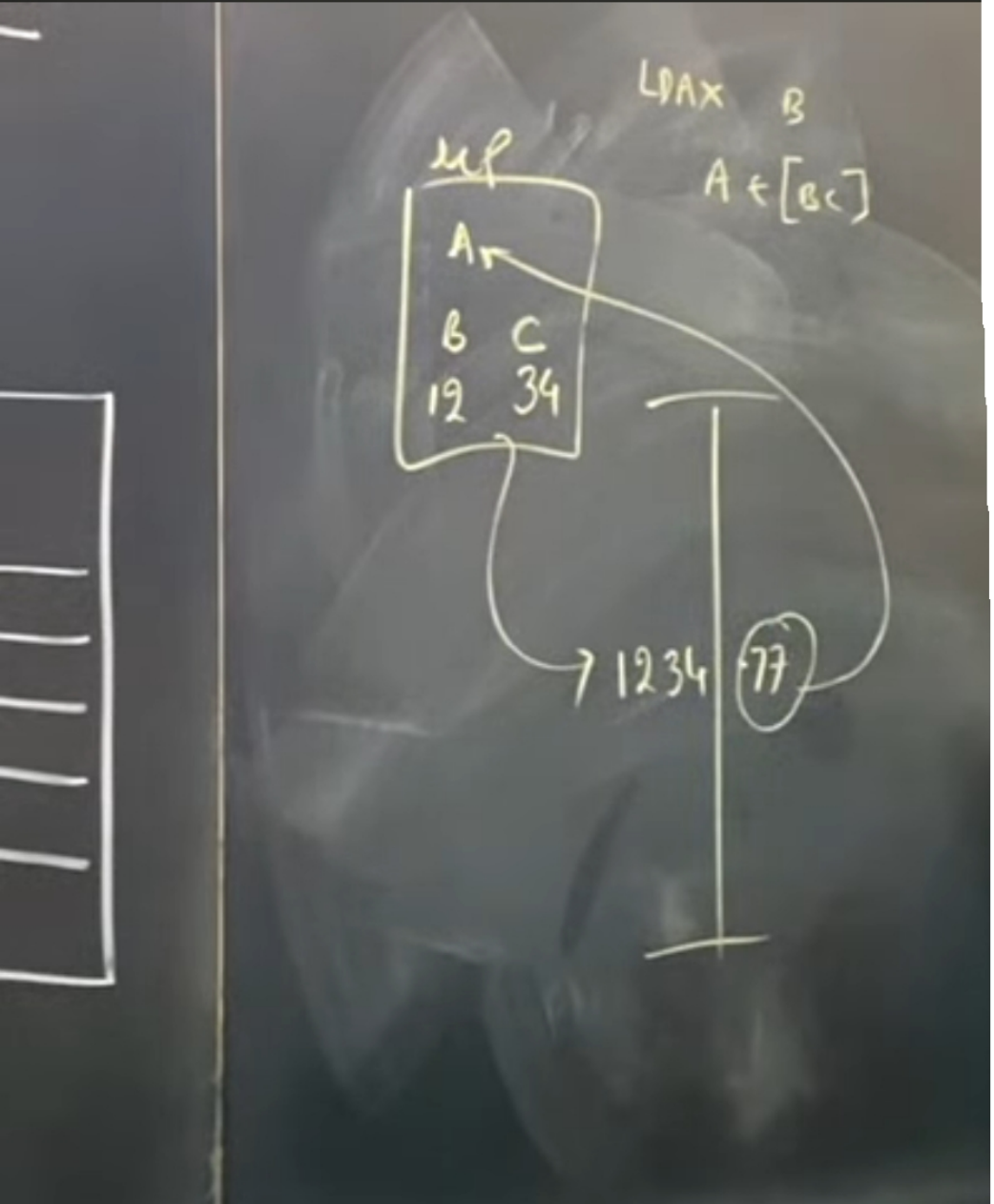


These instr. will store data into Accumulator that is present at given mem. location.

Instructions are always going to be **of 3 bytes** here.



Load contents of Memory Location stored in register Pair BC.



LDAX B

-LDA means Load Accumulator
-X represents pair BC.
-But BC pair contains 16 bits of data and Acc. is of 8-bits

-Thus, the instruction means Load Accumulator with the data stored at Memory location pointed by BC pair.
- B & C are containing the data itself but it will be treated as mem. loc. for this instruction.

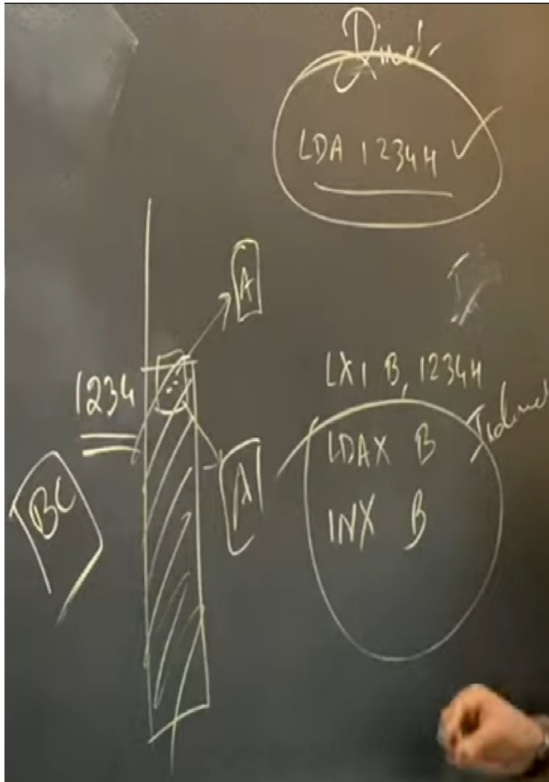
Why do we use Indirect addr. mode when we have Direct addr. mode ?

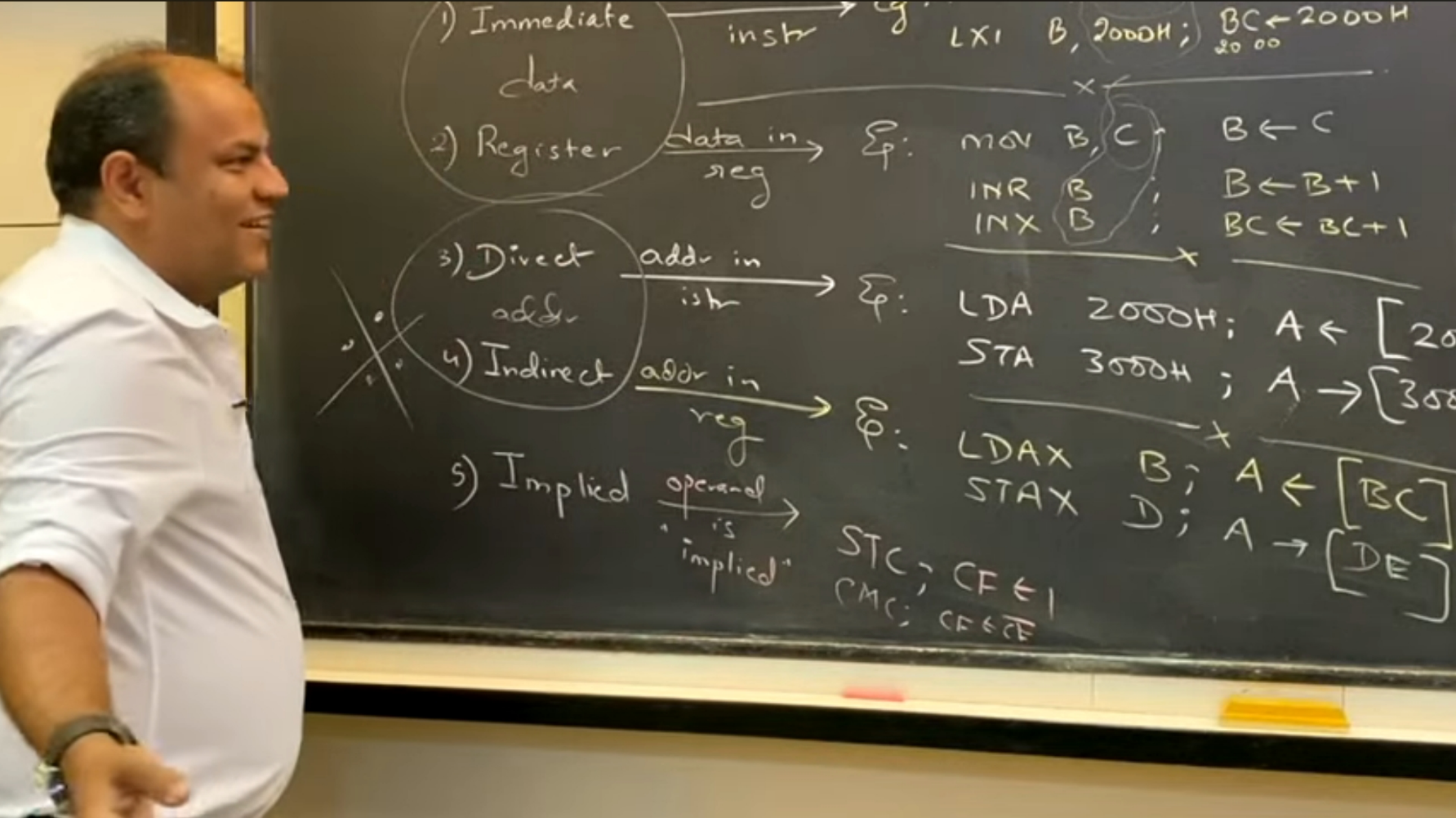
- We store initial memory location in any Register pair and then can apply a loop on it.

- This is simply not possible with Direct addressing mode.

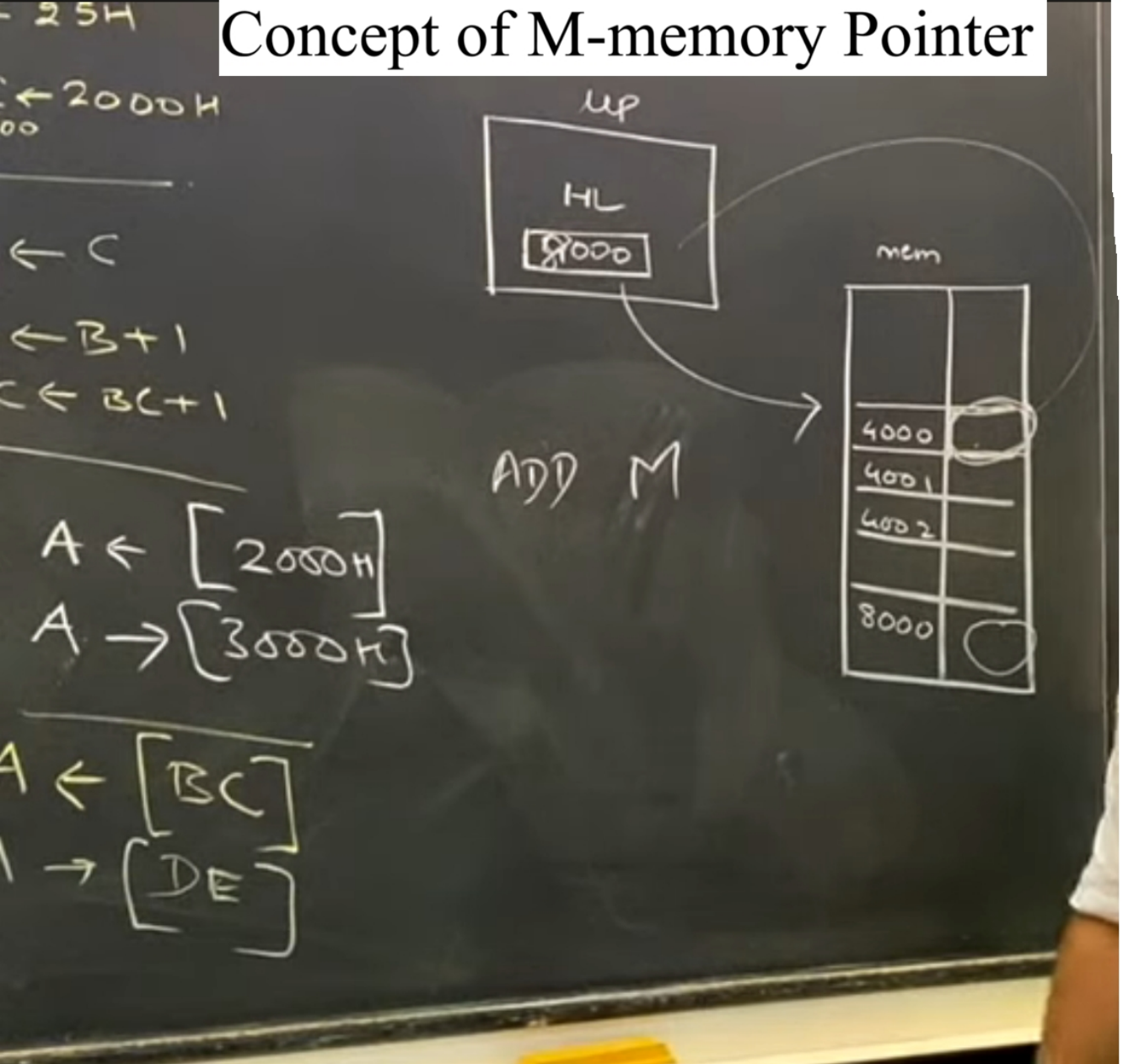
- Assume we want to access a block of 100 mem. locations starting from location 1234H

-What we can do is : Initialize BC pair with 1234 H (LXI 1234H)
- Load Accumulator with the data stored at 1234H (LDAX B) and then increment the BC pair (INX B) to point to new location.
- Run this code for 100 times and our data will be copied.




1) Immediate data
2) Register
3) Direct addr
4) Indirect
5) Implied
data in reg
addr in instr
addr in reg
operand is "implied"
Eg: LXI B, 2000H; BC ← 2000H
Eg: MOV B, C ; B ← C
INR B ; B ← B+1
INX B ; BC ← BC+1
Eg: LDA 2000H; A ← [20
STA 3000H ; A → [300
Eg: LDAX B; A ← [BC]
STAX D; A → [DE]
STC; CF ← 1
CMC; CF ← CF

Concept of M-memory Pointer



M - is the data at a memory location currently pointed by HL pair.

- M would be of 8-bits
- Address of M would be of 16 bits.